



Complete Project Documentation

A comprehensive guide for beginners, staff, and technical teams

Version 1.0 · June 2026 · Confidential

Document	Airfinder Project Documentation
Version	1.0
Date	24 June 2026
Classification	Internal — Confidential
Prepared by	Airfinder Engineering Team
Audience	All staff, new joiners, technical team

Table of Contents

1. Introduction & Project Overview

- › What is Airfinder?
- › Who is this for?
- › Key Features at a Glance

2. Getting Started — For New Joiners

- › How to Access the System
- › Logging In
- › Navigating the Staff Portal
- › Your First Day Checklist

3. Customer-Facing Product Guide

- › The Homepage
- › Searching for Flights
- › AI-Powered Search
- › Multi-City Search
- › Flight Results Page
- › Booking a Flight
- › Multi-City Booking
- › Booking Confirmation & Email
- › Flight Status Tracker
- › Customer Account

4. Staff Portal — User & Account Management

- › Staff Roles Explained
- › Logging into the Staff Portal
- › Dashboard Overview
- › Managing Staff Accounts
- › Managing Customer Accounts
- › Managing Bookings
- › Finance & Revenue Reports

5. System Architecture & Technical Reference

- › Technology Stack
- › Project File Structure
- › Database Models
- › API Reference
- › Authentication & Security
- › Pricing Engine
- › Flight Search Engine
- › Email Service

6. Setup & Deployment Guide

- › Prerequisites
- › Local Development Setup
- › Environment Variables
- › Database Initialisation
- › Running the Server
- › Production Deployment (Render)
- › Gmail SMTP Configuration

7. Security & Compliance

- › Rate Limiting
- › JWT Token Security
- › Password Policy
- › Role-Based Access Control
- › Security Logging
- › CORS Policy

8. Troubleshooting & FAQs

- › Common Login Issues
- › Booking Errors
- › Server Startup Problems
- › Email Not Sending
- › Frequently Asked Questions

Introduction & Project Overview

1.1 What is Airfinder?

Airfinder is a full-featured flight booking and travel management platform built for travel agencies and corporate travel teams. It allows customers to search, compare, and book flights across 191 airports worldwide, while giving staff a powerful back-office portal to manage bookings, customers, revenue, and team accounts — all in one place.

The platform is live at **<http://localhost:5000>** in development and can be deployed to Render or any cloud provider for production use.

1.2 Who is this Document For?

Audience	What They Will Find
New joiners & beginners	Plain-English walkthrough of every feature and page, no technical knowledge needed
Customer support staff	How to help customers with bookings, accounts, and flight status
Administrators & agents	How to manage staff, customers, bookings, and the system day-to-day
Finance staff	Revenue reports, commission tracking, pricing overview
Developers & IT	Architecture, API reference, deployment, and configuration guide

1.3 Key Features at a Glance

- Flight search across 191 airports and 50 airlines with accurate Haversine-based duration estimates
- AI-powered natural language search — type "flight from Lagos to London next month" and it understands
- Multi-city itinerary builder — up to 6 legs in a single booking session
- Flexible dates grid — compare prices across a 7-day window
- Real-time price lock countdown and seat urgency indicators during booking
- Flight status tracker with deterministic mock data (gate, terminal, delay, progress bar)
- Branded booking confirmation emails with full itinerary details
- Staff portal with role-based access (Super Admin, Admin, Agent, Finance)
- Staff account management: create, edit, delete, force password change on first login
- Revenue dashboard: total revenue, commissions, service fees, booking counts

- JWT authentication with bcrypt password hashing for customers and staff
- Rate limiting on all public endpoints to prevent abuse
- Security event logging for failed logins and suspicious activity
- EUR currency display throughout (converted client-side)
- Fully responsive — works on desktop, tablet, and mobile

Getting Started — For New Joiners

Welcome to the team! This chapter is written for people who are new to Airfinder. You do not need any technical background to follow along. By the end of this chapter you will know how to log in, find your way around the staff portal, and help your first customer.

2.1 How to Access the System

Airfinder runs as a web application. You access it through your web browser — Google Chrome or Microsoft Edge are recommended. Your system administrator will give you the web address (URL) to use. In development and internal testing the address is:

```
http://localhost:5000
```

If you are accessing a live production version, you will be given a different URL by your administrator. Always use the URL you were given — do not guess or modify it.

2.2 Logging In

There are two separate login pages — one for customers (the public) and one for staff. As a staff member you must always use the **Staff Portal login**.

Who	Login Page	URL
Customers	Customer Login	/auth/login.html
All Staff	Staff Portal Login	/admin/login.html

First-Time Login

When your account is created by an administrator, you will receive an email containing a temporary password. On your very first login, the system will immediately redirect you to a **Change Password** page. This is mandatory — you cannot use any other part of the portal until you set your own password. Your new password must be at least 8 characters long.

Note: If you try to navigate directly to the dashboard before changing your password, the system will redirect you back to the change-password page automatically. This is a security feature, not a bug.

Forgot Your Password?

On the staff login page, click **Forgot Password**. Enter your email address. If the email is registered, you will receive a reset link valid for 1 hour. If email is not configured, the reset link will be shown directly on the page — copy it and open it in your browser.

2.3 Navigating the Staff Portal

After logging in you will see the main dashboard. On the left side is the navigation sidebar. The items you can see depend on your role — not all staff see all menu items.

Sidebar Item	What It Does	Who Can See It
Dashboard	Overview of bookings, revenue, and customers	All staff
Bookings	View and manage all customer bookings	All staff
Customers	View customer accounts and their history	Super Admin, Admin, Agent
Staff	Create and manage staff accounts	Super Admin, Admin
Finance	Revenue reports and commission summaries	Super Admin, Admin, Finance
Settings	System configuration	Super Admin only

Note: If you do not see a menu item listed above, it means your role does not have permission to access it. Contact your Super Admin if you believe you need access.

2.4 Your First Day Checklist

#	Task	Done?
1	Log in to the Staff Portal at /admin/login.html	■
2	Change your temporary password when prompted	■
3	Confirm your name and role appear correctly in the sidebar	■
4	Review the Dashboard — note total bookings and revenue	■
5	Open the Bookings page and view a recent booking	■
6	Ask your manager which tasks are assigned to your role	■
7	Note the customer login URL to share with customers if needed	■

Customer-Facing Product Guide

This chapter walks through every part of the website that customers interact with. Understanding the customer journey is essential for support staff — you need to know exactly what a customer sees and does so you can help them effectively.

3.1 The Homepage (/)

The homepage is the first thing customers see. It has a large search form at the centre with tabs for three search types: **One Way**, **Round Trip**, and **Multi-City**. Below the search form, the page shows **Popular Routes** — real live prices pulled dynamically from the search engine. These routes include Lagos–London, Dubai–Tokyo, New York–Paris, and many more, and the prices update every time the page loads.

The navbar at the top has links to Flight Status, Login, and Register. Logged-in customers also see a link to their Account Dashboard.

3.2 Standard Flight Search

How to Search

Customers fill in the search form with the following fields:

Field	Description	Required?
From	Departure airport — type 3-letter IATA code or city name	Yes
To	Arrival airport — type 3-letter IATA code or city name	Yes
Departure Date	Date of travel in YYYY-MM-DD format	Yes
Return Date	For round trips — optional	No
Passengers	Number of passengers, 1–9	Yes
Cabin Class	Economy, Business, or First	Yes

After clicking **Search Flights**, they are taken to the **Results Page**. Searches are rate-limited to 60 per minute per IP to prevent abuse.

3.3 AI-Powered Natural Language Search

Customers can describe their trip in plain English instead of filling in form fields. They click the **AI Search** button on the homepage and type something like:

"I need a flight from Lagos to London next month for 2 people in business class"

"Cheapest flight Lagos to Dubai under \$800"

"Flight from Abuja to Paris on July 15"

The AI search parser extracts the origin, destination, date, passenger count, cabin class, and optional budget from the natural language query, then runs a normal flight search with those parameters. AI search is limited to 30 requests per minute.

Tip: If a customer says the AI search did not work, ask them to be more specific about the city names and include a date. Ambiguous queries like "fly somewhere warm" will not produce results.

3.4 Multi-City Search

The Multi-City tab on the homepage allows customers to plan trips with up to 6 separate flight legs. For example: Lagos → London → Paris → New York. Each leg has its own origin, destination, and date. After searching, each leg shows its own list of available flights.

3.5 Flight Results Page (/results.html)

The results page shows all available flights for the searched route and date. Each result card shows the airline, flight number, departure and arrival times, duration, cabin class, and total price in EUR.

Live Features on Results

- **Live timestamp:** Shows when the results were last fetched
- **Auto-refresh:** Results refresh automatically every 5 minutes
- **Price filter:** Customers can filter results by maximum budget
- **Flexible dates grid:** Shows the cheapest price for ± 3 days around the searched date
- **Sort options:** Sort by price or departure time

Urgency Indicators

To assist with purchasing decisions, the results page shows:

- **Price lock countdown:** A 10-minute timer that indicates how long the displayed price is considered valid
- **Seat urgency:** A warning if fewer than 5 seats appear available at that price

Note: These indicators are for user experience purposes. The underlying flight data is generated by a deterministic mock engine — prices shown are consistent for the same route and date combination.

3.6 Booking a Flight (/booking.html)

When a customer clicks **Book Now** on a result card, they are taken to the booking page. If they are not logged in, they will be redirected to login first. The booking page has three steps:

Step	What Happens
1. Passenger Details	Customer enters first name, last name, date of birth, passport number, and nationality for each passenger
2. Extras	Customer selects baggage option (carry-on free, 1 checked bag +€32, 2 checked bags +€55) and seat preference (standard free, window +€14, aisle +€9, extra legroom +€41, front row +€28)
3. Review & Pay	Customer sees full price breakdown and clicks Confirm Booking

On confirmation, a booking reference is generated (format: **AF** + 8 alphanumeric characters, e.g. AFYNLTZG20).

3.7 Multi-City Booking (/multicity.html)

After a multi-city search, the customer sees results for each leg. They select one flight per leg, then click **Book All Legs**. The booking creates one booking record per leg, all linked by a shared **Group Reference**. A single confirmation email shows all legs together.

3.8 Booking Confirmation & Email

After a successful booking, the customer is taken to the **Confirmation Page** (/confirmation.html) which displays their booking reference, flight details, passenger list, and price breakdown.

Simultaneously, the system sends a branded **HTML confirmation email** to the customer's registered email address. The email contains:

- Booking reference number
- Full flight details (route, airline, flight number, date, cabin class)
- Passenger names
- Itemised price breakdown (base fare, markup, service fee, baggage, seat)
- Total amount paid in EUR

For multi-city bookings, the email shows all legs with a combined total and a single group reference for the entire itinerary.

Note: If MAIL_PASSWORD is not configured in the server environment, emails are written to a local log file instead of being sent. Confirm with your administrator whether live email is enabled.

3.9 Flight Status Tracker (/flight-status.html)

Customers (and staff) can look up the real-time status of any flight by entering the flight number and travel date. The tracker shows:

- Current status: On Time / Delayed / Boarding / Departed / Arrived / Cancelled
- Gate and terminal information
- Scheduled and actual departure/arrival times

- Delay duration in minutes (if delayed)
- Aircraft type
- Baggage belt number (if arrived)
- 4-step visual progress bar
- Route with origin and destination city names

The page also includes 8 quick-lookup examples so customers can try it immediately. The tracker supports URL parameters: **?flight=LH401&date;=2026-06-24** — useful for sharing a specific flight status link.

Note: Flight status data is deterministic mock data — the same flight number and date always returns the same status. This ensures a consistent experience while the platform is not yet connected to a live airline data feed.

3.10 Customer Account

Registered customers have access to three account pages:

Page	URL	What It Contains
Dashboard	/account/dashboard.html	Welcome message, booking count, recent bookings, quick links
My Bookings	/account/bookings.html	Full list of all bookings with reference, route, date, status, and price
Profile	/account/profile.html	Update name, phone number, and change password

Customers can cancel a booking from the My Bookings page. Cancelled bookings remain visible in the list with a "cancelled" status.

Chapter 4

Staff Portal — User & Account Management

The Staff Portal is the back-office control centre for Airfinder. This chapter covers every function available to staff, with step-by-step instructions for common support and management tasks.

4.1 Staff Roles Explained

Airfinder uses a 4-level role hierarchy. Higher roles have all the permissions of lower roles, plus additional capabilities.

Role	Level	Key Permissions
Super Admin	4 (Highest)	Full access. Create/edit/delete any staff. Access all data. System settings.
Admin	3	Create and manage agents and finance staff. View all bookings and customers. Cannot delete staff.
Agent	2	View bookings and customers. Update booking status. Cannot manage staff.
Finance	1 (Lowest)	View Dashboard and Bookings. Access Finance revenue reports only. No customer or staff management.

Note: A staff member can only manage accounts with a lower role than their own. An Admin cannot create or edit another Admin. Only a Super Admin can manage Admins.

4.2 Logging into the Staff Portal

Navigate to `/admin/login.html` and enter your email and password.

Staff JWT tokens expire after **12 hours**. If you are redirected to the login page mid-session, your session has expired — simply log in again.

Default Super Admin Credentials

Email	admin@airfinder.com
Password	Admin@2024!

Note: Change the default Super Admin password immediately in any production environment. The default credentials are publicly known and must not be used on live systems.

4.3 Dashboard Overview (/admin/dashboard.html)

The dashboard shows four headline stat cards and two data tables.

Stat Card	What It Shows	Who Sees It
Total Bookings	All bookings ever created	All staff
Confirmed	Bookings with confirmed status	All staff
Total Revenue	Sum of all confirmed booking totals in EUR	All staff
Total Customers	Count of all registered customer accounts	All staff
Staff Members	Total active staff accounts	Super Admin, Admin only

The **Recent Bookings** table shows the 8 most recent bookings with reference, route, total, and status. Click **View All** to go to the full bookings list.

The **Recent Customers** table shows the 8 most recently registered customers. This is only visible to Super Admin, Admin, and Agent roles — Finance staff see only the stat cards and recent bookings.

4.4 Managing Staff Accounts (/admin/staff.html)

Available to: **Super Admin** and **Admin**.

4.4.1 Creating a New Staff Account

Click **+ Add Staff Member** in the top-right of the Staff page. Fill in the form:

Field	Notes
First Name	Required
Last Name	Required
Email	Must be unique. Staff will log in with this email
Role	Select from: Agent, Finance (Admin can assign these). Super Admin can also assign Admin
Status	Active or Inactive
Temporary Password	Optional — if left blank, a 12-character random password is generated automatically

After clicking **Create Staff**, a success panel shows the email address and temporary password. **Copy this immediately** — it is shown only once. A credentials email is also sent to the new staff member.

The new account has **must_change_password = True** set automatically. The staff member will be forced to change their password on first login.

4.4.2 Editing a Staff Account

Click the **Edit** button on any staff row. The edit modal pre-fills all current values. You can update:

- First name and last name
- Email address (must be unique)
- Role (subject to your own role level)
- Status (Active / Inactive) — inactive staff cannot log in

Click **Save Changes**. The table updates immediately with a green confirmation toast.

4.4.3 Resetting a Staff Password

Click **Reset PW** on any staff row. A new 12-character random temporary password is generated, the staff account is flagged for password change on next login, and a credentials email is sent. The new temporary password is also shown in a toast notification on screen.

Tip: Use password reset when a staff member is locked out or has forgotten their password.

4.4.4 Deleting a Staff Account

Super Admin only. Click the **Delete** button (red) on a staff row. A confirmation dialog will appear. Confirm to permanently delete the account. This action cannot be undone.

Note: A Super Admin cannot delete their own account — the backend rejects this with "Cannot delete your own account".

4.4.5 Deactivating vs. Deleting

Action	Effect	When to Use
Deactivate (set Inactive)	Staff cannot log in but account and history preserved	Temporary leave, role change, pending review
Delete	Account permanently removed from database	Account no longer needed, data cleanup

4.5 Managing Customer Accounts (/admin/customers.html)

Available to: **Super Admin, Admin, Agent.**

The Customers page lists all registered customer accounts with their name, email, registration date, booking count, and total spend.

You can search customers by name or email using the search bar at the top.

Common Support Tasks

Task	How to Do It
Find a customer account	Use the search bar with their name or email
See all bookings for a customer	Click on the customer row to view their booking history
Check if account is active	Look at the Status column — Active = green badge, Inactive = grey

4.6 Managing Bookings (/admin/bookings.html)

All staff can view the bookings list. Agents and above can update booking status.

Booking Status Values

Status	Meaning	Badge Colour
Confirmed	Booking is active and paid	Green
Pending	Booking created but not yet confirmed	Amber
Cancelled	Booking was cancelled by customer or staff	Red
Refunded	Booking has been refunded	Blue

Changing a Booking Status

Open a booking by clicking on its row. In the detail view, use the Status dropdown to change the status and click Save. The change is recorded immediately.

Multi-City Bookings

Multi-city bookings have a **Group Reference** that links all legs together. In the bookings list, legs are shown individually but share the same group reference. When filtering or searching, you can search by the group reference to see all legs of a multi-city itinerary.

4.7 Finance & Revenue Reports (/admin/finance.html)

Available to: **Super Admin, Admin, Finance.**

The Finance page provides a revenue summary including:

- Total revenue from all confirmed bookings
- Total commissions earned (3% of base fare by default)
- Total service fees collected (€15 per booking by default)
- Monthly revenue breakdown chart
- Booking count by status

Revenue Configuration

Revenue calculations use these configurable rates (set in .env):

Setting	Default Value	Description
DEFAULT_MARKUP_PERCENT	8%	Markup added on top of base fare
DEFAULT_SERVICE_FEE_USD	\$15	Flat fee per booking
DEFAULT_COMMISSION_PERCENT	3%	Commission on base fare (agency income)

Chapter 5

System Architecture & Technical Reference

This chapter is written for developers and technical staff. It covers the technology stack, codebase structure, database models, and API reference.

5.1 Technology Stack

Layer	Technology	Version / Notes
Backend Framework	Python Flask	3.0.3
Database	SQLite (dev) / PostgreSQL (prod)	Via SQLAlchemy 3.1.1
ORM	Flask-SQLAlchemy	3.1.1
Authentication	PyJWT + bcrypt	JWT HS256, bcrypt hashing
Email	Flask-Mail + Gmail SMTP	TLS port 587
Rate Limiting	Flask-Limiter	3.8.0
CORS	Flask-CORS	4.0.1
Frontend	Vanilla HTML / CSS / JavaScript	No framework, CommonJS modules
Fonts	Inter (Google Fonts)	Loaded from CDN
Production Server	Gunicorn	22.0.0
Deployment	Render	render.yaml configuration

5.2 Project File Structure

```
Airfinder/  
■■■ backend/ # All Python server code  
■ ■■■ app.py # Flask app factory + route mounting  
■ ■■■ config.py # All config from .env  
■ ■■■ extensions.py # Flask-Limiter, Flask-Mail instances  
■ ■■■ middleware/  
■ ■ ■■■ jwt_guard.py # JWT decode decorators  
■ ■ ■■■ role_guard.py # Role-based access decorator
```

- ■■■ models/
- ■■■ database.py # SQLAlchemy db instance + init_db()
- ■■■ user.py # Customer User model
- ■■■ staff.py # Staff model + StaffRole enum
- ■■■ booking.py # Booking model + BookingStatus enum
- ■■■ routes/
- ■■■ auth_customer.py # Customer register/login/reset/profile
- ■■■ auth_staff.py # Staff login/change-password/reset
- ■■■ flights.py # Search, AI search, multicity, status
- ■■■ bookings.py # Create/view/cancel bookings
- ■■■ admin.py # Dashboard, customer list, booking mgmt
- ■■■ staff_mgmt.py # Staff CRUD (create/edit/delete/reset)
- ■■■ services/
- ■■■ mock_flights.py # 191-airport flight search engine
- ■■■ ai_search.py # Natural language query parser
- ■■■ pricing.py # Fare calculation with markup/fees
- ■■■ email_service.py # HTML email templates + send functions
- ■■■ security_logger.py# Failed login / security event logging
- frontend/ # All HTML/CSS/JS (served by Flask)
- ■■■ index.html # Homepage
- ■■■ results.html # Search results
- ■■■ booking.html # Booking flow
- ■■■ confirmation.html # Booking confirmed
- ■■■ multicity.html # Multi-city results + booking
- ■■■ flight-status.html # Flight tracker
- ■■■ auth/ # Customer auth pages
- ■■■ account/ # Customer account pages
- ■■■ admin/ # Staff portal pages
- ■■■ css/ # main.css, admin.css, components.css, search.css
- ■■■ js/ # api.js, auth.js, admin.js, locale.js, search.js
- .env # Environment variables (never commit)
- requirements.txt # Python dependencies
- render.yaml # Render deployment config
- run.py # Entry point (imports create_app)

5.3 Database Models

5.3.1 User (customers)

Column	Type	Description
id	String(36)	UUID primary key
email	String(255)	Unique, indexed, lowercase
password_hash	String(255)	bcrypt hash
first_name / last_name	String(100)	Customer name
phone	String(30)	Optional phone number
is_active	Boolean	False = cannot log in
is_verified	Boolean	Email verification flag (auto-true in demo)
reset_token	String(255)	Password reset token (urlsafe, 32 bytes)
reset_token_expiry	DateTime	Token expires after 15 minutes
created_at / updated_at	DateTime	Audit timestamps

5.3.2 Staff

Column	Type	Description
id	String(36)	UUID primary key
email	String(255)	Unique, indexed, lowercase
password_hash	String(255)	bcrypt hash
first_name / last_name	String(100)	Staff name
role	Enum(StaffRole)	super_admin / admin / agent / finance
must_change_password	Boolean	True = forced to change PW on next login
is_active	Boolean	False = cannot log in
created_by	String(36)	FK to staff.id — who created this account
last_login	DateTime	Recorded on each successful login
reset_token	String(64)	Staff password reset token
reset_token_expires	DateTime	Expires after 1 hour

5.3.3 Booking

Column	Type	Description
id	String(36)	UUID primary key
reference	String(20)	Unique booking ref e.g. AFYNLTZG20
user_id	String(36)	FK to users.id
flight_id	String(100)	Flight identifier from search engine
origin / destination	String(10)	IATA airport codes
departure_date	String(20)	Date as stored at booking time
airline / flight_number	String	Snapshot at booking time
cabin_class	String(20)	economy / business / first
passengers_json	Text	JSON array of passenger objects
passenger_count	Integer	Count of passengers
base_fare_usd	Float	Raw fare before markup
markup_usd	Float	8% markup (configurable)
service_fee_usd	Float	\$15 flat fee (configurable)
baggage_fee_usd / seat_fee_usd	Float	Optional add-on fees
total_usd	Float	Final total paid
commission_usd	Float	3% of base fare (agency revenue)
status	Enum(BookingStatus)	pending / confirmed / cancelled / refunded
group_reference	String(20)	Links multi-city legs together
is_multicity	Boolean	True for multi-city booking legs

5.4 API Reference

All API endpoints are prefixed with their blueprint URL. Authentication uses **Bearer token** in the Authorization header.

5.4.1 Customer Authentication (/api/auth)

Method	Endpoint	Auth	Description
POST	/register	None	Create customer account
POST	/login	None	Customer login → JWT token
POST	/forgot-password	None	Send password reset email
POST	/reset-password	None	Reset password with token
GET	/me	JWT	Get current customer profile
PUT	/me	JWT	Update name and phone
POST	/change-password	JWT	Change own password

5.4.2 Staff Authentication (/api/staff/auth)

Method	Endpoint	Auth	Description
POST	/login	None	Staff login → 12h JWT token
POST	/change-password	JWT	Change temp password (bypasses must_change_password guard)
GET	/me	Staff JWT	Get current staff profile
POST	/forgot-password	None	Send reset link to staff email
POST	/reset-password	None	Reset with token (1h expiry)

5.4.3 Flight Search (/api/flights)

Method	Endpoint	Rate Limit	Description
GET	/search	60/min	Standard flight search (origin, destination, date, passengers, cabin, return_date, budget_max)
POST	/search/ai	30/min	Natural language search — body: {query}
POST	/search/multicity	20/min	Multi-leg search — body: {legs[], passengers, cabin}
GET	/search/flexible	20/min	Price grid ± 3 days around a date
GET	/featured	None	Popular routes with live prices
GET	/airports	None	Full list of 191 airports
POST	/pricing/calculate	60/min	Calculate total for given base fare + options
GET	/pricing/options	None	Baggage and seat fee options
GET	/status	60/min	Flight status by number + date

5.4.4 Bookings (/api/bookings)

Method	Endpoint	Auth	Description
POST	/	JWT	Create single booking (10/min limit)
POST	/multicity	JWT	Create multi-city booking (5/min limit)
GET	/	JWT	List bookings (customers see own; staff see all)
GET	/	JWT	Get single booking
POST	//cancel	JWT	Cancel a booking

5.4.5 Admin (/api/admin)

Method	Endpoint	Role	Description
GET	/dashboard	All staff	Stats: bookings, revenue, customers, staff
GET	/customers	Admin+	Paginated customer list with search
GET	/bookings	All staff	Paginated booking list with status filter
PUT	/bookings/	Agent+	Update booking status or notes
GET	/finance/summary	Finance+	Revenue summary and breakdown

5.4.6 Staff Management (/api/admin/staff)

Method	Endpoint	Role	Description
GET	/	Admin+	List all staff accounts
POST	/	Admin+	Create staff (optional temp_password field)
GET	/	Admin+	Get single staff account
PUT	/	Admin+	Edit name, email, role, or status
POST	//reset-password	Admin+	Generate new temp password + email
DELETE	/	Super Admin	Permanently delete staff account

5.5 Authentication & Security Architecture

JWT Token Design

Both customers and staff authenticate with JSON Web Tokens (JWT). Tokens are HS256-signed with the JWT_SECRET from .env. Token payload:

Claim	Type	Description
user_id	String	UUID of the user or staff account
role	String	customer / super_admin / admin / agent / finance
must_change_password	Boolean	True = staff must change PW before other actions
exp	Integer	Expiry: customers 24h, staff 12h

Password Hashing

All passwords are hashed with bcrypt before storage. The raw password is never stored and cannot be recovered — only verified with bcrypt.checkpw(). When a staff password is reset, a new bcrypt hash of the temporary password is stored.

Role Guard

The @require_role(*roles) decorator on backend routes verifies the JWT and checks the role claim. Requests with an insufficient role return HTTP 403.

5.6 Pricing Engine (backend/services/pricing.py)

Every booking total is calculated by the pricing engine. The formula:

```
markup = base_fare × markup_pct / 100 (default 8%)
```

```
baggage_fee = baggage_rate × passengers
```

```
seat_fee = seat_rate × passengers
```

```

subtotal_per_pax = base_fare + markup + service_fee + per_pax_baggage + per_pax_seat
total = subtotal_per_pax * passengers
commission = base_fare * commission_pct / 100 (default 3%)

```

Option	Fee per Passenger
Carry-on baggage	€0 (included)
1 Checked bag	+€32
2 Checked bags	+€55
Standard seat	€0 (included)
Window seat	+€14
Aisle seat	+€9
Extra legroom	+€41
Front row	+€28

5.7 Flight Search Engine (backend/services/mock_flights.py)

The flight search engine generates realistic flight options for any route across 191 airports and 50 airlines. Key features:

- **Haversine formula:** Accurate great-circle distance between airports used to compute realistic flight durations
- **Deterministic seeding:** Same route + date always produces the same set of flights (uses hashlib.md5 seed)
- **Regional airline routing:** 6 regional airline pools; longhaul flag controls which airlines appear on cross-region routes
- **Airport carrier whitelist (AIRPORT_CARRIERS):** Secondary airports like Port Harcourt (PHC) only show airlines that actually serve them
- **lru_cache on _estimate_duration:** Repeated duration lookups for the same route are cached in memory

Airline Routing Logic

When building the airline pool for a route:

- Same region: only carriers from that region are eligible
- Cross-region: longhaul carriers from both regions + all Middle East carriers
- Fallback: if pool has fewer than 4 airlines, any longhaul airline is added
- AIRPORT_CARRIERS whitelist applied last — removes non-eligible airlines for restricted airports

5.8 Email Service

(backend/services/email_service.py)

The email service sends branded HTML emails for four events:

Function	Trigger	Template
send_welcome_email	Customer registers	Welcome + "Start Exploring" CTA
send_booking_confirmation_email	Single booking confirmed	Flight details + itemised receipt
send_multicity_confirmation_email	Multi-city booking confirmed	All legs listed + combined total
send_staff_credentials_email	Staff created or password reset	Email + temporary password

All emails share a consistent branded wrapper: green (#407E3C) header with the Airfinder logo, white content card, and "© 2026 Airfinder" footer.

Dev mode: If MAIL_PASSWORD contains "your-" (i.e. is a placeholder), emails are written to `.claude/email_dev.log` instead of being sent. This prevents accidental sends during development.

Chapter 6

Setup & Deployment Guide

This chapter covers everything needed to get Airfinder running from scratch, configure it for production, and deploy it to a cloud host.

6.1 Prerequisites

Requirement	Version	Notes
Python	3.10+	Earlier versions not tested
pip	Latest	Comes with Python
Git	Any	For cloning and version control
Gmail account	Any	For SMTP email (or use any SMTP provider)

6.2 Local Development Setup

Step 1: Clone the repository

```
git clone https://github.com/Obinwanne1/Airfinder.git
cd Airfinder
```

Step 2: Create and activate a Python virtual environment

```
python -m venv venv
venv\Scripts\activate # Windows
source venv/bin/activate # Mac / Linux
```

Step 3: Install dependencies

```
pip install -r requirements.txt
```

Step 4: Create the .env file (see Section 6.3)

Step 5: Run the server

```
python -m backend.app
```

Step 6: Open in browser

```
http://localhost:5000
```

Note: On Windows, always use **python -m backend.app** (not `python backend/app.py`) to ensure the module paths resolve correctly. Set `PYTHONPATH` if needed: `$env:PYTHONPATH = "C:\path\to\Airfinder"`

6.3 Environment Variables (.env)

Create a file named **.env** in the project root. Never commit this file to git.

Variable	Example Value	Description
SECRET_KEY	a-very-long-random-string	Flask session secret — use 32+ random chars
JWT_SECRET	another-random-string	JWT signing key — use 32+ random chars
DATABASE_URL	sqlite:///airfinder.db	SQLite for dev; postgresql:// for prod
PORT	5000	Server port
MAIL_SERVER	smtp.gmail.com	SMTP host
MAIL_PORT	587	SMTP port (TLS)
MAIL_USE_TLS	True	Always True for Gmail TLS
MAIL_USERNAME	you@gmail.com	Gmail address
MAIL_PASSWORD	xxxx xxxx xxxx xxxx	Gmail App Password (16 chars, spaces ok)
MAIL_DEFAULT_SENDER	Airfinder	From: header in emails
DEFAULT_MARKUP_PERCENT	8	Fare markup percentage
DEFAULT_SERVICE_FEE_USD	15	Flat service fee per booking
DEFAULT_COMMISSION_PERCENT	3	Commission on base fare
SUPER_ADMIN_EMAIL	admin@airfinder.com	Auto-created super admin email
SUPER_ADMIN_PASSWORD	Admin@2024!	Auto-created super admin password
ALLOWED_ORIGINS	http://localhost:5000	CORS origins (comma-separated)

6.4 Database Initialisation

The database is created automatically on first run. SQLAlchemy calls `db.create_all()` inside `init_db()` when the Flask app starts. The SQLite file is stored at **instance/airfinder.db**.

To seed demo data (optional):

```
python seed_demo.py
```

To reset the super admin password if locked out:

```
python reset_admin.py
```

6.5 Running the Server

Development (with auto-reload)

```
python -m backend.app
```

Production with Gunicorn

```
gunicorn "backend.app:create_app()" --bind 0.0.0.0:5000 --workers 4
```

6.6 Production Deployment on Render

The project includes a **render.yaml** file for one-click deployment on Render.

- Push the repository to GitHub
- Create a new Web Service on Render and connect the GitHub repo
- Render reads render.yaml and configures build and start commands automatically
- Add all .env variables as Environment Variables in the Render dashboard
- For PostgreSQL: create a Render PostgreSQL database, copy the connection string, set DATABASE_URL
- Deploy — Render will install requirements and start Gunicorn

Note: Change SECRET_KEY and JWT_SECRET to long random strings in production. Never use the development defaults on a live server.

6.7 Gmail SMTP Configuration

To send real emails, you need a Gmail App Password (not your regular Gmail password).

Steps to create a Gmail App Password:

Step	Action
1	Go to myaccount.google.com → Security
2	Enable 2-Step Verification if not already on
3	Search for "App Passwords" in the search bar
4	Select App: Mail, Device: Other → type "Airfinder" → Generate
5	Copy the 16-character password shown (with or without spaces)
6	Set MAIL_PASSWORD=xxxx xxxx xxxx xxxx in your .env file
7	Restart the server — MAIL_PASSWORD change requires restart

Tip: Gmail App Passwords look like: abcd efgh ijkl mnop. Spaces are fine in the .env value.

Note: Gmail has a daily sending limit of 500 emails for regular accounts. For high-volume production use, consider SendGrid, Mailgun, or AWS SES.

Security & Compliance

7.1 Rate Limiting

Flask-Limiter applies per-IP rate limits on all sensitive endpoints to prevent brute-force attacks and abuse:

Endpoint	Limit
Customer login	10 per minute
Staff login	10 per minute
Customer registration	10 per hour
Forgot password (customer)	5 per hour
Forgot password (staff)	5 per minute
AI search	30 per minute
Multi-city search	20 per minute
Standard flight search	60 per minute
Create booking	10 per minute
Create multi-city booking	5 per minute

7.2 JWT Token Security

- All tokens are signed with HS256 using the JWT_SECRET environment variable
- Tokens are short-lived: 24 hours for customers, 12 hours for staff
- Tokens are stored in localStorage on the client (acceptable for internal tools; for public-facing apps, consider httpOnly cookies)
- Token claims include role and must_change_password — both checked on every protected route
- No token revocation mechanism currently — if a token must be invalidated immediately, the JWT_SECRET can be rotated (this logs everyone out)

7.3 Password Policy

Rule	Applies To
Minimum 8 characters	All users and staff
Cannot reuse current password when changing	All users and staff
bcrypt hashing (work factor ≥ 12)	All stored passwords
Temporary passwords are 12 chars with letters, digits, and symbols	Staff accounts
Must change temporary password on first login	All staff accounts
Password reset tokens expire in 15 min (customers) / 1 hour (staff)	Forgot password flow

7.4 Role-Based Access Control (RBAC)

Access control is enforced at two levels:

Backend (authoritative): The `@require_role()` decorator on every protected route verifies the JWT role claim server-side. No client-side check can bypass this.

Frontend (UX only): The data-role-only HTML attribute hides sidebar items and UI elements for unauthorised roles. This improves UX but is not a security boundary.

Note: Never rely on frontend role checks for security. The backend is the only authoritative access control layer.

7.5 Security Event Logging

Failed authentication attempts are logged to **security.log** in the project root:

```
{ "event": "login_failed", "user_type": "customer", "email": "...", "ip": "...",  
  "timestamp": "..." }
```

Events logged:

- login_failed — customer or staff login with wrong credentials
- password_change_failed — wrong current password provided

Review security.log regularly in production. Multiple failed login attempts from the same IP may indicate a brute-force attack (rate limiting will block most automated attacks, but manual review is still good practice).

7.6 CORS Policy

Cross-Origin Resource Sharing is configured to allow API requests only from origins listed in ALLOWED_ORIGINS (.env). The default is localhost:5000 only.

For production, update ALLOWED_ORIGINS to your actual domain:

```
ALLOWED_ORIGINS=https://airfinder.yourdomain.com
```

Note: CORS only restricts browser-based requests. Direct API calls from tools like curl or Postman are not blocked by CORS — backend authentication is the real security boundary.

Troubleshooting & FAQs

8.1 Common Login Issues

Symptom	Likely Cause	Fix
"Invalid credentials"	Wrong email or password	Ask admin to reset password via Staff page → Reset PW button
"Account deactivated"	is_active = False on the account	Admin sets Status to Active in Edit modal
Redirected to change-password immediately	must_change_password = True	Normal — change the temporary password to proceed
Session expired mid-use	Staff token expired (12h limit)	Log out and log back in
Login page loops (redirects back)	Must_change_password and navigating away from change-password page	Complete the password change — do not navigate away

8.2 Booking Errors

Error	Cause	Fix
"Required fields: ..."	Frontend did not send all required booking fields	Reload the booking page and try again
Booking created but no email	MAIL_PASSWORD is a placeholder	Configure Gmail App Password in .env and restart server
Duplicate reference collision (very rare)	Random reference already exists	The system auto-retries — this resolves itself

8.3 Server Startup Problems

Error	Cause	Fix
"ModuleNotFoundError: No module named backend"	Running python backend/app.py directly	Run: python -m backend.app (with PYTHONPATH set to project root)
"Address already in use"	Port 5000 is occupied by another process	Run: netstat -ano findstr :5000 → taskkill /F /PID
"No module named flask"	Virtual environment not activated	Run: venv\Scripts\activate then retry

8.4 Email Not Sending

Work through this checklist in order:

Check	How
1. Is MAIL_PASSWORD set?	Open .env — check MAIL_PASSWORD is not the placeholder "your-gmail-app-password-here"
2. Is it a Gmail App Password?	Must be a 16-char App Password, not your regular Gmail password — 2FA must be enabled
3. Server restarted after .env change?	Kill the server and start it again — running process does not pick up .env changes
4. Check email_dev.log	If dev mode is active, emails go to .claude/email_dev.log — open it to see what would have been sent
5. Is Gmail account blocking access?	Check Gmail → Security → Recent security activity for blocked sign-in attempts

8.5 Frequently Asked Questions

Can a customer cancel their own booking?

Yes. From /account/bookings.html, the customer can cancel any of their bookings. The status changes to "cancelled" and the booking remains visible in their history.

What happens if a staff member forgets their temporary password?

An Admin or Super Admin can reset it from the Staff page → Reset PW button. A new temporary password is generated and emailed to the staff member. It is also shown in the success toast.

Can I change the markup or service fee?

Yes — update `DEFAULT_MARKUP_PERCENT` and `DEFAULT_SERVICE_FEE_USD` in the `.env` file and restart the server. All new bookings will use the new rates. Existing bookings are not affected (prices are stored as a snapshot at booking time).

Why do all searches return results even for unusual routes?

The flight search engine is a mock system that generates realistic options for any valid airport pair. It is not connected to live airline inventory. This means any two airports in the 191-airport database will always return results.

Can a Finance user see customer details?

No. Finance role only has access to the Dashboard (bookings + revenue stats) and the Finance page. The Customers nav link is hidden, and the `/api/admin/customers` endpoint returns 403 for Finance tokens.

How do I add a new staff member without them receiving an email?

If email is not configured, the credentials email will silently fail but the account is still created. The temporary password is shown in the success modal on the Staff page immediately after creation — copy it and share it manually with the new staff member.

Is the flight data real?

No. All flight data — prices, times, availability, flight numbers, and status — is generated by a deterministic mock engine. The same search always returns the same results for a given route and date. This is intentional for the current version of the platform. Connecting to a live flight data API (e.g. Amadeus, Skyscanner) would replace the `mock_flights.py` service.

What databases are supported?

SQLite is used for local development (no setup required). PostgreSQL is used for production. The `DATABASE_URL` environment variable controls which is used. Any SQLAlchemy-compatible database works.

How do I add a new airport?

Open `backend/services/mock_flights.py` and add the airport to the `AIRPORTS` dictionary with its IATA code, name, city, country, region, latitude, and longitude. If the airport should have a carrier whitelist (e.g. a regional airport with limited airlines), add it to `AIRPORT_CARRIERS` with the list of eligible airline codes.